

Module 4 — GenAI Strategies

Model selection, multimodal, and guardrails

What this module covers

- Build a **Research Assistant** web app — streaming chat, tool use, vision, and audio.
- Model selection: choosing the right model for **quality, cost, and latency**.
- Multimodal: **vision**/image analysis, **speech-to-text** (gpt-transcribe).
- Guardrails: schema validation, content filters, and **confidence thresholds**.

🎯 Learning goals

- Build a **FastAPI app** with SSE streaming chat and tool integration.
- Connect an **MCP server** to a web API for real-time tool use.
- Count tokens, enforce **budgets**, and truncate context gracefully.
- Work with **multimodal** inputs: images (GPT-6 Luna), audio (gpt-transcribe).
- Chain **guardrails**: schema check → content filter → confidence gate.



Guardrails — defence in depth

Schema validation, content filters, confidence gates

Guardrails — defence in depth

- **Schema validation**: does the response parse into the expected shape?
- **Content filter**: block or flag toxic, off-topic, or hallucinated content.
- **Confidence threshold**: reject low-confidence answers with a fallback.
- **Chain them**: schema → filter → confidence, in order. Fail early.

Guardrail chain

```
def run_guardrails(response, schema, blocked_terms, min_confidence):
    # Step 1: Schema check
    try:
        data = schema.model_validate_json(response)
    except ValidationError as e:
        return GuardrailResult(passed=False, reason=f"Schema: {e}")

    # Step 2: Content filter
    for term in blocked_terms:
        if term.lower() in response.lower():
            return GuardrailResult(passed=False, reason=f"Blocked: {term}")

    # Step 3: Confidence gate
    if data.confidence < min_confidence:
        return GuardrailResult(passed=False, reason="Low confidence")

    return GuardrailResult(passed=True, data=data)
```

- → Each guardrail returns early on failure — no wasted work
- → Structured result lets the caller decide what to do next

Unguarded vs guarded pipeline

No guardrails	Guardrail chain
Parse errors crash the app	Invalid schema → retry or fallback
Toxic content reaches users	Blocked content → safe rejection
Hallucinated data treated as fact	Low confidence → "I'm not sure"
No fallback path	Every failure is loggable and testable



Model selection + tokens

Choosing the right model and managing budgets

⚖️ Model selection trade-offs

- **Quality**: GPT-6 Sol for hard reasoning; GPT-6 Luna for simple classification.
- **Cost**: 10-50x price difference between tiers — measure before committing.
- **Latency**: smaller models respond faster; streaming helps perception.
- **Context window**: GPT-6 Luna takes ~1M tokens in (128k out) — but every token is billed, so trim anyway.
- Rule of thumb: start with the cheapest model that passes your eval suite.

Token counting and budgets

```
import tiktoken

enc = tiktoken.get_encoding("o200k_base")  # all current OpenAI chat models

def count_tokens(text: str) -> int:
    return len(enc.encode(text))

def enforce_budget(messages, max_tokens):
    """Drop oldest user turns until under budget."""
    total = sum(count_tokens(m["content"]) for m in messages)
    while total > max_tokens and len(messages) > 2:
        removed = messages.pop(1)  # keep system prompt
        total -= count_tokens(removed["content"])
    return messages
```

- → tiktoken gives exact counts for OpenAI models
- → Always preserve the system prompt — it sets behaviour



Multimodal — see and hear

Vision with GPT-6 Luna and audio with gpt-transcribe

Multimodal inputs

- **Vision**: send images as base64 data URIs or public URLs in user messages.
- **Audio input**: the transcription API (gpt-transcribe) turns speech into text.
- Same Responses API call — content is just a **list of parts** (`input_text`, `input_image`) instead of a string.
- Use `client.responses.parse(text_format=ImageAnalysis)` to get typed vision output.

Vision — image analysis

```
response = client.responses.create(
    model=MODEL,
    reasoning={"effort": "none"},
    input=[{
        "role": "user",
        "content": [
            {"type": "input_text", "text": "Describe what you see in this image."},
            {"type": "input_image", "image_url": f"{img_b64}"},
        ],
    }],
)
```

- → Content is a list of parts — text and images mixed freely
- → Works with base64 data URIs or public HTTPS URLs

Audio — speech-to-text

```
import io

audio_file = io.BytesIO(audio_bytes)
audio_file.name = "recording.wav"

transcript = client.audio.transcriptions.create(
    model="gpt-transcribe",
    file=audio_file,
)
print(transcript.text)  # "Hello, this is a voice memo about..."
```

- The transcription API accepts file-like objects — wrap raw bytes in BytesIO
- Set .name so the API knows the audio format



Demo — Multimodal

Switch to terminal: python demo/demo.py — Parts 1–3



Putting it all together

Summary and exercises

Summary

Stream the chat from FastAPI

SSE delivers tokens as they arrive. An MCP server supplies the tools.

Count tokens before you send

Surprise truncation is worse than deliberate trimming.

Images and audio are inputs too

Vision and speech-to-text sit beside the text chat.

Chain guardrails, never skip them

Schema, then content filter, then confidence. Fail fast, log always.

Exercises — Build the Research Assistant

01 — Research chat: streaming chat with MCP tool calling (server provided)

02 — Multimodal: add vision (image analysis) and audio (speech-to-text)



Module 4 — Complete

Next: RAG fundamentals